Academic Paper Pipeline: Open-Source Python Libraries Guide

> **Goal:** Fetch academic PDFs → Extract text → Chunk intelligently → Store in a vector database → Send to LLM for classification
> **Everything below is open-source and free to use.**

---

1. The Full Pipeline at a Glance

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
┌──────────────┐
│  FETCH PDFs  │──▶│ EXTRACT TEXT │──▶│   CHUNK      │──▶│  STORE IN    │──▶│
SEND TO     │
│  from APIs   │    │  from PDFs   │    │  the text    │    │  VECTOR DB   │    │
LLM         │
└──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘
└──────────────┘

   arxiv            PyMuPDF4LLM        Chonkie /          ChromaDB /
Ollama /
   paperscraper     Unstructured       LangChain          FAISS
Claude API
   OpenAlex         PyMuPDF            LlamaIndex         Qdrant
OpenAI
   Unpaywall        GROBID             Semantic
HuggingFace
```

---

2. FETCHING Academic Papers (PDFs)

These libraries let you search for papers and download their PDFs.

2.1 `arxiv` (Python wrapper for arXiv API)

```bash
pip install arxiv
```

```python
```

```python
import arxiv

client = arxiv.Client()
search = arxiv.Search(
    query="transformer efficiency pruning",
    max_results=10,
    sort_by=arxiv.SortCriterion.Relevance
)

for paper in client.results(search):
    print(paper.title)
    print(paper.summary)          # abstract
    print(paper.pdf_url)          # direct PDF link
    paper.download_pdf(dirpath="./pdfs")  # downloads the PDF!
```

Best for: CS, Physics, Math papers (all open-access on arXiv).

2.2 `paperscraper` (Multi-source: arXiv, PubMed, bioRxiv, chemRxiv)

```bash
pip install paperscraper
```

```python
from paperscraper.pubmed import get_and_dump_pubmed_papers
from paperscraper.pdf import save_pdf

# Search PubMed
get_and_dump_pubmed_papers(
    ["transformer", "efficiency"],
    output_filepath="results.jsonl"
)

# Download a PDF by DOI
paper_data = {"doi": "10.48550/arXiv.2207.03928"}
save_pdf(paper_data, filepath="paper.pdf")
```

Best for: Multi-source searching (PubMed, bioRxiv, medRxiv, chemRxiv, arXiv).

2.3 `pyalex` (OpenAlex API — what your project already uses)

```bash
pip install pyalex
```

```python
from pyalex import Works

papers = (
    Works()
    .search("efficient attention mechanisms")
    .filter(publication_year=">2022", type="article")
    .get()
)

for paper in papers:
    print(paper["title"])
    print(paper["doi"])  # use DOI to fetch PDF from Unpaywall
```

2.4 Unpaywall (Free legal PDF access via DOI)

```python
import requests

doi = "10.1234/example.doi"
email = "your@email.com"
resp = requests.get(f"https://api.unpaywall.org/v2/{doi}?email={email}")
data = resp.json()

if data.get("best_oa_location"):
    pdf_url = data["best_oa_location"]["url_for_pdf"]
    print(f"Free PDF at: {pdf_url}")
```

Best for: Finding open-access versions of any paper with a DOI.

---

3. EXTRACTING Text from PDFs

Once you have PDFs, you need to extract clean text.

3.1 `PyMuPDF4LLM` ⭐ (Best for LLM pipelines)

```bash
pip install pymupdf4llm
```

```python
```

```python
import pymupdf4llm

# Extract as Markdown (preserves headings, tables, structure)
md_text = pymupdf4llm.to_markdown("paper.pdf")

# Extract with page-level chunks + metadata
chunks = pymupdf4llm.to_markdown(
    "paper.pdf",
    page_chunks=True,        # split by page
    write_images=True,       # also extract images
    image_path="images/",
    image_format="png"
)

for chunk in chunks:
    print(f"Page {chunk['metadata']['page']}: {chunk['text'][:200]}...")
```

Why it's great: Outputs clean Markdown that LLMs understand well. Handles tables, multi-column layouts, and embedded images.

3.2 `Unstructured` (Document ETL for LLMs)

```bash
pip install unstructured "unstructured[pdf]"
```

```python
from unstructured.partition.pdf import partition_pdf

elements = partition_pdf(filename="paper.pdf")

for element in elements:
    print(f"[{element.category}] {element.text[:200]}")
    # Categories: Title, NarrativeText, ListItem, Table, etc.
```

Output element types:

- `Title` — section headings
- `NarrativeText` — body paragraphs
- `ListItem` — bullet points
- `Table` — table content
- `Image` — detected images

Why it's great: Understands document layout — knows the difference between a heading, a paragraph, and a table.

3.3 `PyMuPDF` (Fast, low-level PDF extraction)

```bash
pip install pymupdf
```

```python
import fitz  # PyMuPDF

doc = fitz.open("paper.pdf")
for page in doc:
    text = page.get_text()
    print(text)
```

3.4 `GROBID` (Machine-learning based academic PDF parser)

GROBID is a Java server that uses ML to parse academic papers into structured XML — it understands title, authors, abstract, sections, references, etc.

```bash
# Run GROBID server via Docker
docker run --rm -p 8070:8070 lfoppiano/grobid:0.8.1
```

```python
import requests

# Send PDF to GROBID
with open("paper.pdf", "rb") as f:
    resp = requests.post(
        "http://localhost:8070/api/processFulltextDocument",
        files={"input": f}
    )
# Returns structured TEI XML with sections, references, etc.
print(resp.text)
```

Best for: When you need precise section-level parsing of academic papers.

Comparison Table: PDF Extraction

Library	Speed	Layout Aware	Tables	Academic Structure	Ease of Use
PyMuPDF4LLM	Fast	✅ Good	✅ Yes	❌ No	⭐ Easy
Unstructured	Medium	✅ Great	⚠️ Okay	❌ No	⭐ Easy
PyMuPDF	Fastest	❌ Basic	❌ No	❌ No	⭐ Easy
GROBID	Slow	✅ Great	✅ Yes	✅ Yes (sections!)	⚠️ Needs Docker

---

4. CHUNKING the Text

Chunking = splitting extracted text into smaller pieces that fit in an LLM's context window while preserving meaning.

Why Chunk?

- LLMs have context limits (4K–128K tokens)
- Embedding models have smaller limits (~512 tokens)
- Smaller, focused chunks = better retrieval accuracy
- Chunks that are too large dilute attention; too small lose context

4.1 Chunking Strategies (from simple to advanced)

Strategy 1: Fixed-Size Chunking (simplest)

```python
def fixed_size_chunk(text, chunk_size=500, overlap=50):
    """Split text into fixed-size chunks with overlap."""
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i + chunk_size])
        chunks.append(chunk)
    return chunks

chunks = fixed_size_chunk(extracted_text, chunk_size=500, overlap=50)
```

Pros: Simple, predictable size.
Cons: Cuts mid-sentence, mid-paragraph — destroys context.

Strategy 2: Recursive / Separator-Based (recommended starting point)

Splits by natural boundaries: `\n\n` (paragraphs) → `\n` (lines) → `. ` (sentences) → ` ` (words).

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,       # max characters per chunk
    chunk_overlap=200,     # overlap between chunks
    separators=["\n\n", "\n", ". ", " ", ""]
)

chunks = splitter.split_text(extracted_text)
for i, chunk in enumerate(chunks):
    print(f"Chunk {i}: {len(chunk)} chars — {chunk[:100]}...")
```

Strategy 3: Semantic Chunking (best quality)

Groups sentences by meaning using embeddings — splits where the topic changes.

```python
from chonkie import SemanticChunker

chunker = SemanticChunker(
    embedding_model="minishlab/potion-base-8M",
    chunk_size=512,
    similarity_threshold=0.5   # lower = more splits
)

chunks = chunker.chunk(extracted_text)
for chunk in chunks:
    print(f"[{chunk.token_count} tokens] {chunk.text[:150]}...")
```

Strategy 4: Section-Aware Chunking (best for academic papers)

If you used GROBID or Unstructured, you already have section boundaries:

```python
```

```python
from unstructured.partition.pdf import partition_pdf
from unstructured.chunking.title import chunk_by_title

# Step 1: Parse PDF into elements
elements = partition_pdf(filename="paper.pdf")

# Step 2: Chunk by section titles
chunks = chunk_by_title(
    elements,
    max_characters=1500,
    new_after_n_chars=1000,
    combine_text_under_n_chars=200
)

for chunk in chunks:
    print(f"[{chunk.category}] {chunk.text[:200]}...")
```

4.2 Chunking Libraries Comparison

Library	Install	Best Chunking Methods	Academic-Friendly
Chonkie	`pip install chonkie`	Semantic, Token, Recursive, LLM-based	✅ Great
LangChain	`pip install langchain`	Recursive, Token, Markdown, HTML	✅ Good
LlamaIndex	`pip install llama-index`	SentenceSplitter, SemanticSplitter	✅ Great
Unstructured	`pip install unstructured`	chunk_by_title (layout-aware)	✅ Best
LlamaSherpa	`pip install llmsherpa`	Layout-aware "smart chunking"	✅ Great

4.3 Recommended Chunk Sizes

Use Case	Chunk Size	Overlap
Embedding + vector search (RAG)	256–512 tokens	10–20%
LLM classification (your project)	500–1500 tokens	50–100 tokens
Full paper summary	1000–2000 tokens	200 tokens

5. STORING in a Vector Database

A vector database stores your chunks as embeddings (numerical vectors) so you can search by meaning.

5.1 `ChromaDB` ⭐ (Easiest to start with)

```bash
pip install chromadb
```

```python
```

```python
import chromadb

# Create a persistent database (saved to disk)
client = chromadb.PersistentClient(path="./paper_db")

# Create a collection (like a table)
collection = client.get_or_create_collection(
    name="research_papers",
    metadata={"hnsw:space": "cosine"}  # cosine similarity
)

# Add chunks with metadata
for i, chunk in enumerate(chunks):
    collection.add(
        documents=[chunk.text],
        metadatas=[{
            "paper_title": "FlashAttention",
            "section": "methodology",
            "page": 3,
            "doi": "10.1234/example",
            "year": 2023
        }],
        ids=[f"flash_attention_chunk_{i}"]
    )

# Query: find chunks similar to a question
results = collection.query(
    query_texts=["How does FlashAttention reduce memory usage?"],
    n_results=5
)

for doc, meta, dist in zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
):
    print(f"[Distance: {dist:.3f}] [{meta['paper_title']}] {doc[:150]}...")
```

Key features:

- Auto-embeds text using `all-MiniLM-L6-v2` by default
- Stores on disk or in-memory
- Metadata filtering (filter by year, journal, classification, etc.)
- Simple 4-function API: `add`, `query`, `update`, `delete`

5.2 `FAISS` (Facebook AI Similarity Search — fastest for large datasets)

```bash
pip install faiss-cpu sentence-transformers
```

```python
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# Create embeddings
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(chunk_texts)

# Build FAISS index
dimension = embeddings.shape[1]  # 384
index = faiss.IndexFlatIP(dimension)    # inner product (cosine after normalization)
faiss.normalize_L2(embeddings)
index.add(embeddings)

# Search
query_vec = model.encode(["efficient attention"])
faiss.normalize_L2(query_vec)
distances, indices = index.search(query_vec, k=5)

# Save / Load
faiss.write_index(index, "papers.faiss")
index = faiss.read_index("papers.faiss")
```

Best for: Millions of chunks. Pure speed. No metadata filtering built in (you manage that separately).

5.3 Other Options

Database	Install	Hosted Option	Metadata Filter	Best For
ChromaDB	`pip install chromadb`	Chroma Cloud	✅ Yes	Prototyping, small-medium
FAISS	`pip install faiss-cpu`	No	❌ Manual	Speed, large datasets
Qdrant	`pip install qdrant-client`	Yes	✅ Rich	Production, complex filters

Database	Install	Hosted Option	Metadata Filter	Best For
Weaviate	`pip install weaviate-client`	Yes	✅ Yes	Hybrid search
Pinecone	`pip install pinecone`	Yes (paid)	✅ Yes	Fully managed, enterprise

---

6. SENDING Chunks to the LLM

6.1 Direct Classification (what your project does now)

Your current approach: send title + abstract to Ollama → get classification.

```python
# Your existing classifier.py approach (simplified)
payload = {
    "model": "llama3.2",
    "prompt": f"Classify this paper:\nTitle: {title}\nAbstract: {abstract[:800]}",
    "stream": False,
    "temperature": 0
}
response = requests.post("http://localhost:11434/api/generate", json=payload)
```

6.2 Enhanced: Chunk-Based Classification

When a paper is too long for one prompt, send the most relevant chunks:

```python
```

```python
def classify_with_chunks(paper_title, chunks_collection, ollama_url, ollama_model):
    """
    1. Query the vector DB for chunks most relevant to classification
    2. Send those chunks to the LLM
    """
    # Step 1: Retrieve relevant chunks from vector DB
    results = chunks_collection.query(
        query_texts=[
            "main contribution method proposed efficiency scaling"
        ],
        n_results=5,
        where={"paper_title": paper_title}  # filter to this paper
    )

    relevant_text = "\n\n".join(results["documents"][0])

    # Step 2: Send to LLM with the relevant chunks
    prompt = f"""Classify this research paper based on the following excerpts.

Title: {paper_title}

Key Excerpts:
{relevant_text[:3000]}

Classify as "efficiency", "scaling", or "neither".
Reply in JSON: {{"classification": "...", "reason": "...", "accuracy": 0-100}}"""

    payload = {
        "model": ollama_model,
        "prompt": prompt,
        "stream": False,
        "temperature": 0
    }

    response = requests.post(ollama_url, json=payload, timeout=60)
    return response.json()["response"]
```

6.3 RAG (Retrieval-Augmented Generation) Pattern

The full pattern for question-answering over your paper database:

```python
```

```python
def ask_papers(question, collection, ollama_url, ollama_model):
    """
    RAG: Retrieve relevant chunks, then generate an answer.
    """
    # 1. RETRIEVE — find chunks relevant to the question
    results = collection.query(
        query_texts=[question],
        n_results=5
    )

    context_chunks = results["documents"][0]
    sources = results["metadatas"][0]

    # 2. AUGMENT — build a prompt with retrieved context
    context = "\n\n---\n\n".join([
        f"[From: {src['paper_title']}, Page {src.get('page', '?')}]\n{chunk}"
        for chunk, src in zip(context_chunks, sources)
    ])

    prompt = f"""Answer the question using ONLY the provided context.
If the context doesn't contain the answer, say "I don't have enough information."

Context:
{context}

Question: {question}

Answer:"""

    # 3. GENERATE — send to LLM
    payload = {
        "model": ollama_model,
        "prompt": prompt,
        "stream": False,
        "temperature": 0.1
    }

    response = requests.post(ollama_url, json=payload, timeout=60)
    return response.json()["response"]
```

7. SAVING Everything

7.1 Save Raw Data (papers + metadata)

```python
import json
import sqlite3

# Option A: JSON (simple, human-readable)
with open("papers_database.json", "w") as f:
    json.dump(all_papers, f, indent=2)

# Option B: SQLite (structured, queryable)
conn = sqlite3.connect("papers.db")
conn.execute("""
    CREATE TABLE IF NOT EXISTS papers (
        id INTEGER PRIMARY KEY,
        title TEXT,
        abstract TEXT,
        authors TEXT,
        doi TEXT UNIQUE,
        year INTEGER,
        journal TEXT,
        pdf_path TEXT,
        classification TEXT,
        confidence REAL,
        reason TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

conn.execute("""
    CREATE TABLE IF NOT EXISTS chunks (
        id INTEGER PRIMARY KEY,
        paper_id INTEGER,
        chunk_index INTEGER,
        text TEXT,
        section TEXT,
        page INTEGER,
        token_count INTEGER,
        FOREIGN KEY (paper_id) REFERENCES papers(id)
    )
""")
conn.commit()
```

7.2 Save Vector Database (persistent ChromaDB)

```python
# ChromaDB with PersistentClient auto-saves to disk
client = chromadb.PersistentClient(path="./chroma_db")
# Everything you add is automatically persisted
# Next time you open with the same path, all data is there
```

7.3 Save Classification Results

```python
import csv

# Append results as they come in
with open("classification_results.csv", "a", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=[
        "title", "doi", "classification", "confidence",
        "reason", "num_chunks", "model_used", "timestamp"
    ])
    writer.writerow({
        "title": paper["title"],
        "doi": paper["doi"],
        "classification": result["classification"],
        "confidence": result["accuracy"],
        "reason": result["reason"],
        "num_chunks": len(chunks),
        "model_used": "llama3.2",
        "timestamp": datetime.now().isoformat()
    })
```

---

8. COMPLETE EXAMPLE: End-to-End Pipeline

```python
```

```python
"""
Complete pipeline: Fetch → Extract → Chunk → Store → Classify
"""
import arxiv
import pymupdf4llm
import chromadb
import requests
import json
import os

# ── CONFIG ──
OLLAMA_URL = "http://localhost:11434/api/generate"
OLLAMA_MODEL = "llama3.2"
PDF_DIR = "./pdfs"
DB_PATH = "./paper_vectordb"

os.makedirs(PDF_DIR, exist_ok=True)

# ── STEP 1: FETCH papers from arXiv ──
print("Step 1: Fetching papers...")
client_arxiv = arxiv.Client()
search = arxiv.Search(
    query="efficient transformer attention",
    max_results=5,
    sort_by=arxiv.SortCriterion.Relevance
)

papers = []
for result in client_arxiv.results(search):
    pdf_path = result.download_pdf(dirpath=PDF_DIR)
    papers.append({
        "title": result.title,
        "abstract": result.summary,
        "pdf_path": str(pdf_path),
        "doi": result.doi or "N/A",
        "year": result.published.year,
        "authors": ", ".join(a.name for a in result.authors)
    })
    print(f"  Downloaded: {result.title[:60]}...")

# ── STEP 2: EXTRACT text from PDFs ──
print("\nStep 2: Extracting text from PDFs...")
for paper in papers:
    md_text = pymupdf4llm.to_markdown(paper["pdf_path"])
    paper["full_text"] = md_text
    print(f"  Extracted {len(md_text)} chars from {paper['title'][:50]}...")
```

```python
# ── STEP 3: CHUNK the text ──
print("\nStep 3: Chunking text...")
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n## ", "\n### ", "\n\n", "\n", ". ", " "]
)

for paper in papers:
    paper["chunks"] = splitter.split_text(paper["full_text"])
    print(f"  {paper['title'][:50]}... → {len(paper['chunks'])} chunks")

# ── STEP 4: STORE in ChromaDB ──
print("\nStep 4: Storing in ChromaDB...")
chroma_client = chromadb.PersistentClient(path=DB_PATH)
collection = chroma_client.get_or_create_collection("papers")

for paper in papers:
    for i, chunk in enumerate(paper["chunks"]):
        collection.add(
            documents=[chunk],
            metadatas=[{
                "title": paper["title"],
                "doi": paper["doi"],
                "year": paper["year"],
                "chunk_index": i
            }],
            ids=[f"{paper['doi']}_{i}"]
        )
print(f"  Stored {collection.count()} total chunks")

# ── STEP 5: CLASSIFY using relevant chunks ──
print("\nStep 5: Classifying papers...")
results = []

for paper in papers:
    # Retrieve most relevant chunks for classification
    relevant = collection.query(
        query_texts=["main contribution proposed method efficiency scaling"],
        n_results=3,
        where={"title": paper["title"]}
    )

    context = "\n\n".join(relevant["documents"][0]) if relevant["documents"][0] else
```

```
    prompt = f"""Classify this AI/ML paper.

Title: {paper['title']}
Key Excerpts:
{context[:2000]}

Reply in JSON only:
{{"classification": "efficiency" or "scaling" or "neither", "reason": "...", "accura

    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "temperature": 0
    }

    try:
        resp = requests.post(OLLAMA_URL, json=payload, timeout=60)
        raw = resp.json()["response"]
        result = json.loads(raw.replace("```json", "").replace("```", "").strip())
    except Exception as e:
        result = {"classification": "error", "reason": str(e), "accuracy": 0}

    paper["classification"] = result
    print(f"  {paper['title'][:50]}... → {result['classification'].upper()}")
    results.append(paper)

# ── STEP 6: SAVE everything ──
print("\nStep 6: Saving results...")

# Save to JSON (full data)
save_data = [{
    "title": p["title"],
    "doi": p["doi"],
    "year": p["year"],
    "authors": p["authors"],
    "abstract": p["abstract"],
    "num_chunks": len(p["chunks"]),
    "classification": p["classification"]
} for p in results]

with open("pipeline_results.json", "w") as f:
    json.dump(save_data, f, indent=2)
```

```
print("\nDone! Results saved to pipeline_results.json")
print(f"Vector DB saved to {DB_PATH}/")
```

---

9. How the Data Flows to the LLM

```
┌──────────────────────────────────────────────────────────────────────┐
│                      HOW DATA REACHES THE LLM                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  PDF (10,000+ words)                                                 │
│       │                                                              │
│       ▼                                                              │
│  PyMuPDF4LLM extracts → Markdown text                               │
│       │                                                              │
│       ▼                                                              │
│  Chunker splits → 20-50 chunks of ~500 tokens each                  │
│       │                                                              │
│       ▼                                                              │
│  Each chunk → embedded into a 384-dim vector                         │
│       │                                                              │
│       ▼                                                              │
│  Vectors stored in ChromaDB with metadata                            │
│       │                                                              │
│       ▼                                                              │
│  At query time: "classify this paper"                                │
│       │                                                              │
│       ▼                                                              │
│  ChromaDB finds 3-5 most relevant chunks (cosine similarity)         │
│       │                                                              │
│       ▼                                                              │
│  Selected chunks assembled into a prompt:                            │
│       ┌──────────────────────────────────────────┐                   │
│       │ System: You are a classifier...          │                   │
│       │ Context: [chunk 1] [chunk 2] [chunk 3] │  ← only the      │
│       │ Question: Is this efficiency/scaling?   │    relevant      │
│       └──────────────────────────────────────────┘    parts!         │
│       │                                                              │
│       ▼                                                              │
│  LLM reads ~1500 tokens instead of 10,000+                          │
│  → Faster, cheaper, more accurate classification                     │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

10. Quick Install Commands

```bash
# Fetching
pip install arxiv paperscraper pyalex

# PDF Extraction
pip install pymupdf4llm pymupdf

# Chunking
pip install chonkie langchain llama-index

# Vector Database
pip install chromadb

# Embeddings
pip install sentence-transformers

# LLM (local)
# Install Ollama from https://ollama.ai
ollama pull llama3.2
```

---

11. Recommended Stack for Your Project

Given your existing OpenAlex + Ollama setup, here's what I'd recommend adding:

Step	Current	Recommended Addition
Fetch PDFs	OpenAlex (metadata only)	+ `arxiv` lib to download actual PDFs
Extract text	Abstract only	+ `PyMuPDF4LLM` for full paper text
Chunk	None (sends full abstract)	+ `LangChain RecursiveCharacterTextSplitter`
Store	JSON/CSV files	+ `ChromaDB` for vector search
Classify	Ollama (title + abstract)	+ RAG: retrieve relevant chunks first
Save	CSV + JSON	+ SQLite for structured queries

This would let you go from classifying based on abstracts to classifying based on the full paper content — which will dramatically improve classification accuracy for papers where the abstract is vague or doesn't clearly state the contribution.